

INHA UNIVERSITY IN TASHKENT

SOC 2040 SYSTEM PROGRAMMING

Exceptional Control Flow (Chapter 8) PROCESSES

INSTRUCTOR: Jalolliddin Yusupov
PROFESSOR OF COMPUTER SCIENCE & ENG

Lecture Outline

- System Calls
- Processes
- fork()
- Zombies ahead

System Calls

- **Each x86-64 system call has a unique ID number**
- **Examples:** ≠ A process is **not** a "program" and **not** a "processor" — it's the running instance

Number	Name	Description
0	read	Read file
1	write	Write file
2	open	Open file
3	close	Close file
4	stat	Get info about file
57	fork	Create process
59	execve	Execute a program
60	exit	Terminate process
62	kill	Send signal to process

Context switching is the process where a computer's CPU stops running one task, saves its state (context), and loads the saved state of another task to allow multitasking.

A process is an instance of a running program
A **program** is a file on disk. A **process** is what comes to life when the kernel loads that program into memory and starts executing it. The same program can spawn multiple processes (think: opening two terminal windows both running bash).

The kernel makes each process believe two things that aren't literally true:

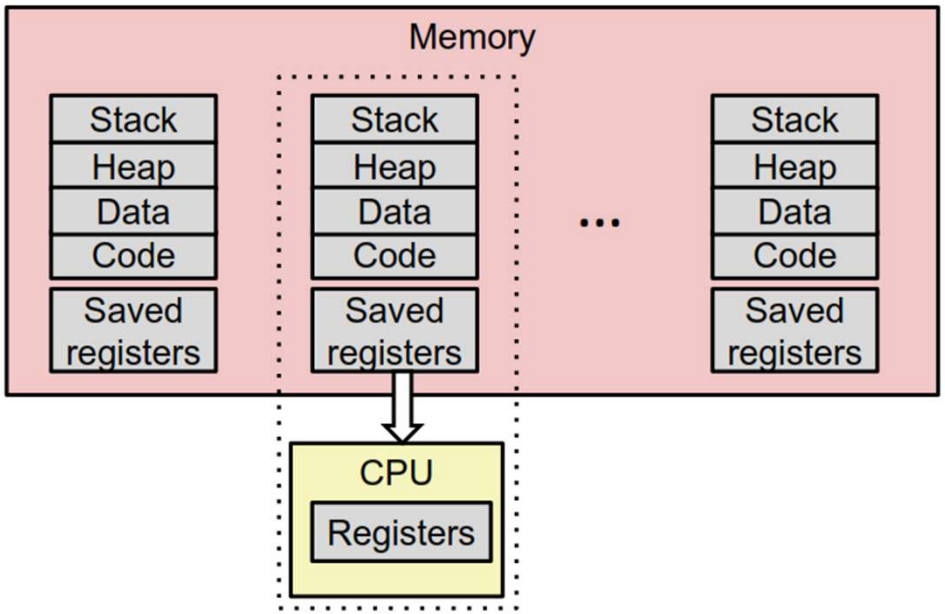
Logical control flow

"I have the CPU all to myself"

Achieved via **context switching** — the kernel rapidly swaps which process runs on the CPU, saving and restoring register state each time.

System Calls

≠ A process is **not** a "program" and **not** a "processor" — it's the running instance



A process is an instance of a running program
A **program** is a file on disk. A **process** is what comes to life when the kernel loads that program into memory and starts executing it. The same program can spawn multiple processes (think: opening two terminal windows both running bash).

The kernel makes each process believe two things that aren't literally true:

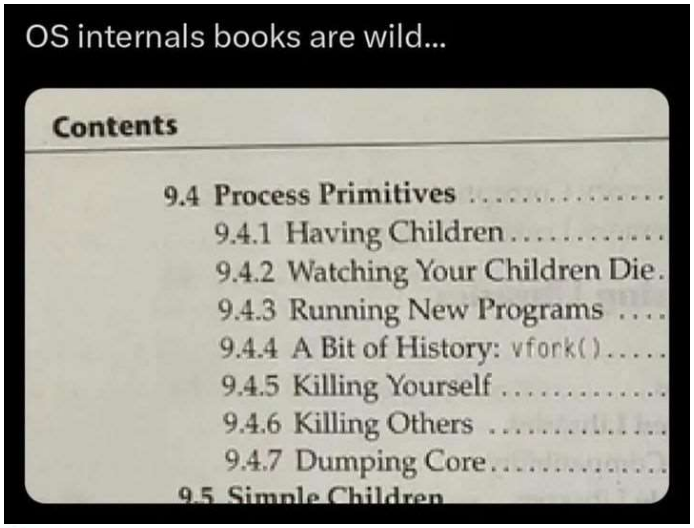
Private address space

"I have memory all to myself"

Achieved via **virtual memory** — each process sees addresses 0x0…0xFFF… but the hardware maps them to different physical frames.

virtual memory is an abstraction layer that makes every running program (process) believe it has its own private, contiguous block of RAM

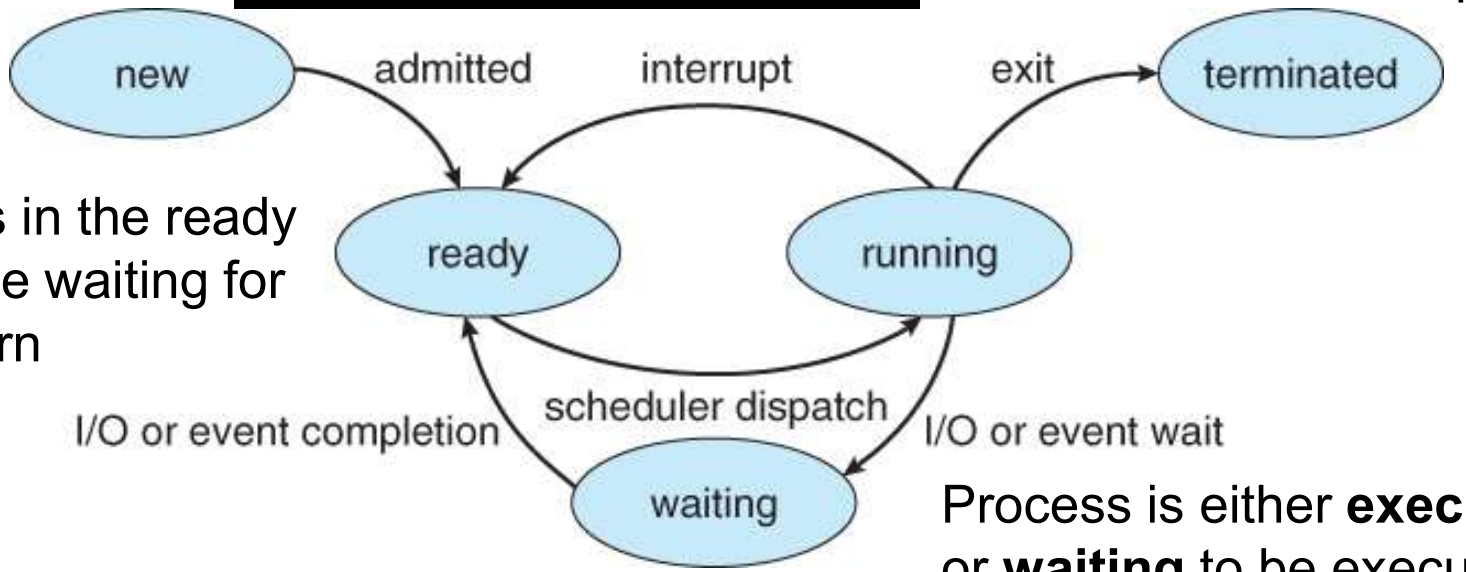
Process State



The process has just been created - **fork()**

(1) receiving a signal
(2) returning from the main routine
(3) calling the exit function

Process is stopped permanently



It sits in the ready queue waiting for its turn

Process is either **executing**, or **waiting** to be executed and will eventually be scheduled (i.e., chosen to execute) by the kernel

Creating Processes

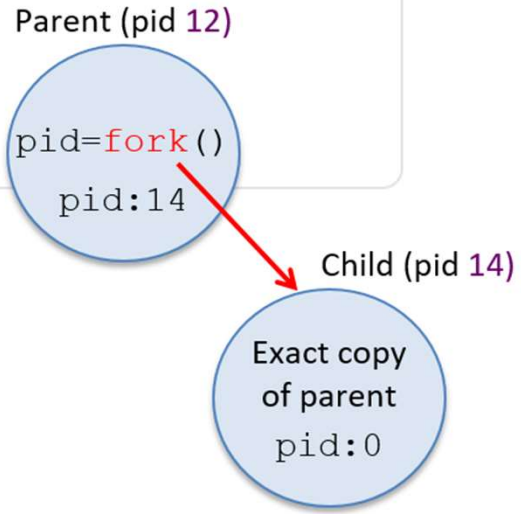
Every process has a PID

Each process gets a unique positive integer ID assigned by the kernel. Two syscalls let you query it:

```
pid_t getpid(void)
```
Returns your own PID

```
pid_t getppid(void)
```
Returns your parent's PID

`pid_t` is defined in `<sys/types.h>` — on Linux it's just an `int`.



fork() = clone the current process

A parent creates a child by calling `fork()`. The child gets an identical (but separate) copy of the parent's address space and open file descriptors, but a different PID.

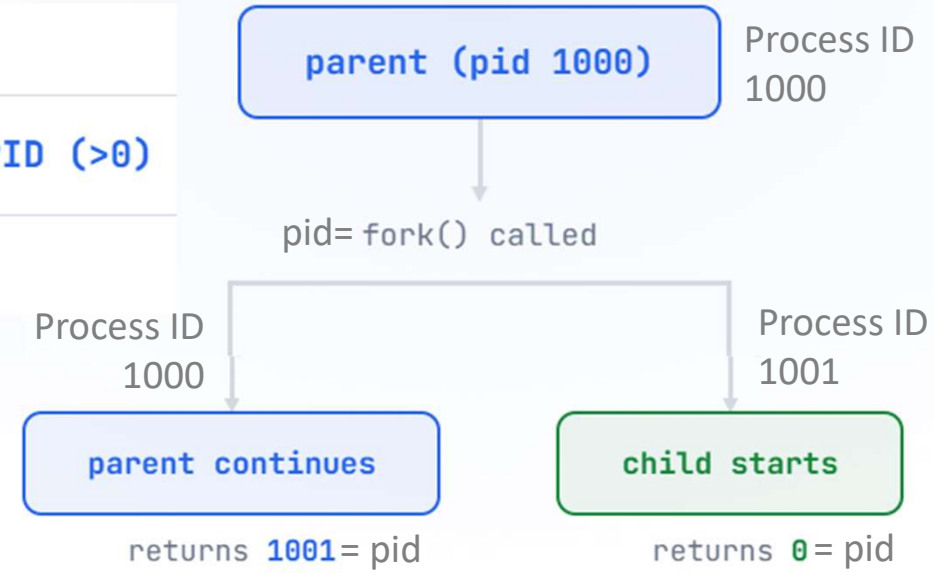
(data, stack)

> **Key insight:** `fork()` is called *once* but returns *twice* — once in each process. The return value tells you which one you are.

The parent and the child are separate processes that run concurrently. Which one first?

Creating Processes

In the **child**	`0`
In the **parent**	`child's PID (>0)`
On **error**	`-1`



If fork() returns -1, the child was not created. The global errno variable tells you why.
This is a hard rule for *all* system calls (except those returning void).

```c
#include <stdio.h>    // For printf()
#include <unistd.h>   // For fork(), getpid(), and getppid()
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        printf("SOME ERROR\n");
        return 1; // Replacing exit(1) to remove need for stdlib.h
    }
    if (pid == 0) {
        printf("child:  my pid = %d, parent = %d\n", getpid(), getppid());
    } else {
        printf("parent: my pid = %d, child = %d\n", getpid(), pid);
    }
    return 0;
}
```

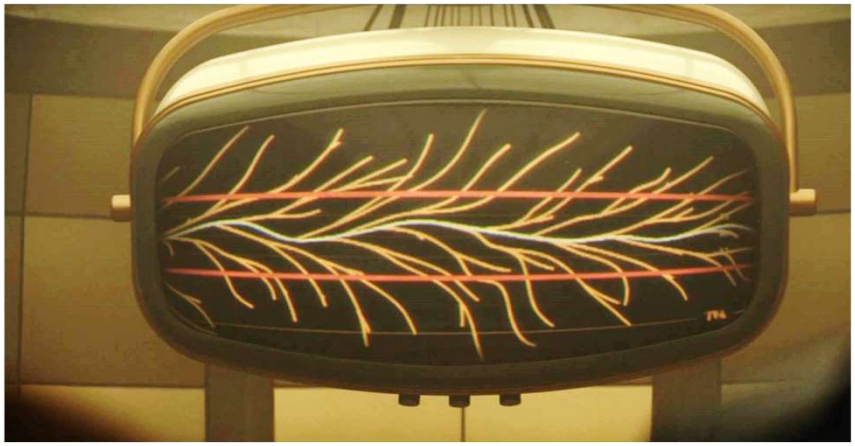
Always check the return value

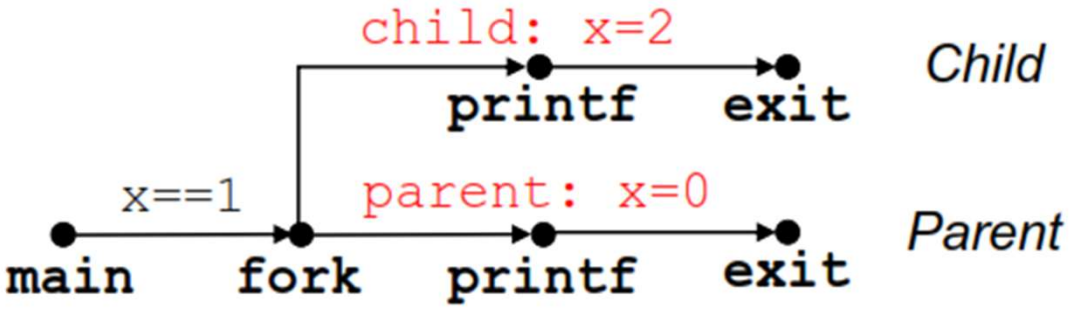
```
gcc fork_example.c -o fork_example
```

Process Graph

```c
#include <stdio.h>   // For printf()
#include <unistd.h>  // For fork(), getpid(), and getppid()
#include <stdlib.h>
int main()
{
    pid_t pid;
    int x = 1;
    pid = fork();
    if (pid == 0)    /* Child */
    {
        printf("child : x=%d\n", ++x);
        exit(0);
    }
    /* Parent */
    printf("parent: x=%d\n", --x);
    exit(0);
}
```

Sketch the process graph, where each horizontal arrow corresponds to a process that executes instructions from left to right, and each vertical arrow corresponds to the execution of a fork function.



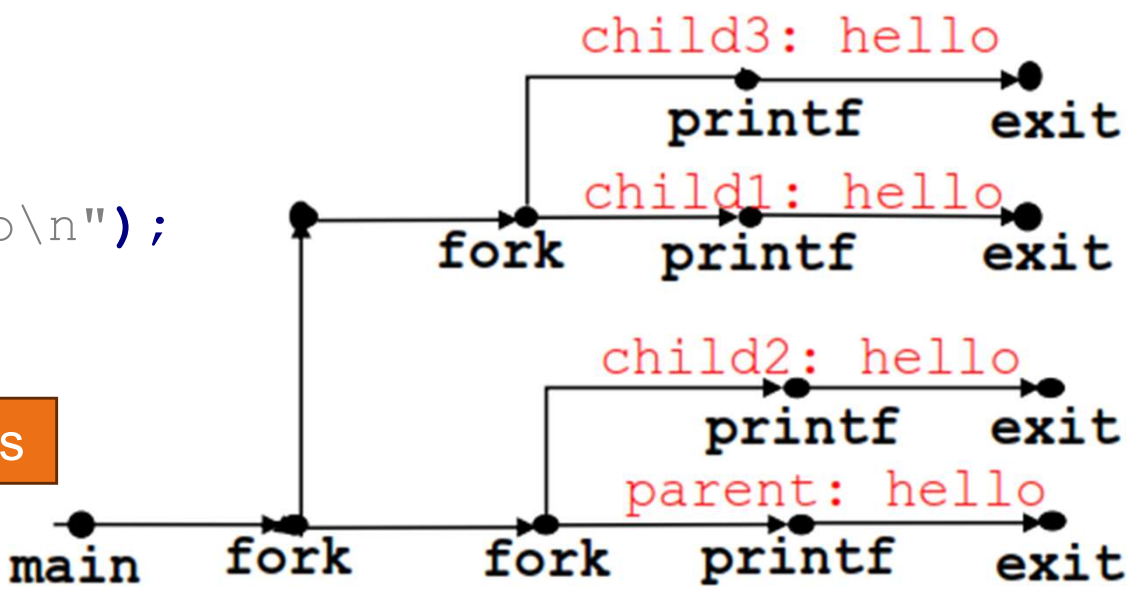


Modeling fork with Process Graphs

Examples of fork programs and their process graphs

```
int main() {
    fork();
    fork();
    printf("hello\n");
    exit(0);
}
```

Prints four output lines




```
CHILD3
printf("hello\n");
exit(0);

CHILD1
fork();
printf("hello\n");
exit(0);

CHILD2
printf("hello\n");
exit(0);
```

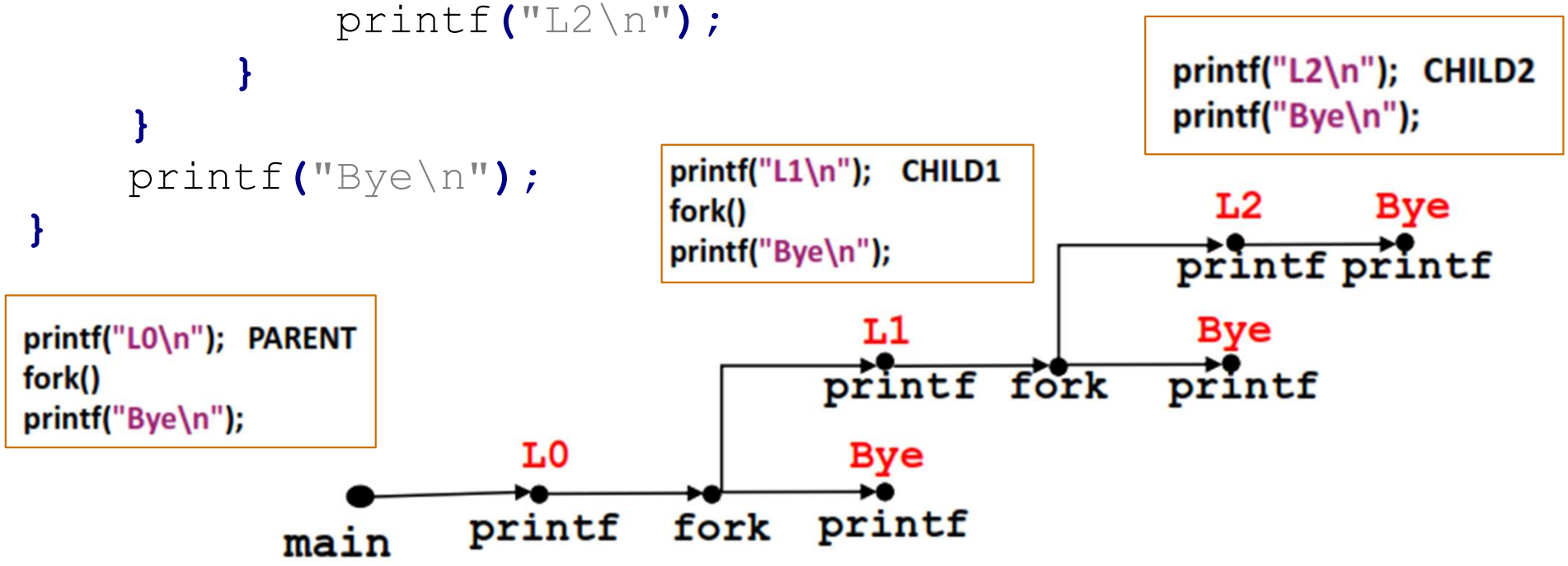
```
fork();   PARENT
fork();
printf("hello\n");
exit(0);
```

HOMEWORK

```
int main() {
    fork();
    fork();
    fork();
    printf("hello\n");
    exit(0);
}
```

fork Example: Nested forks in parent

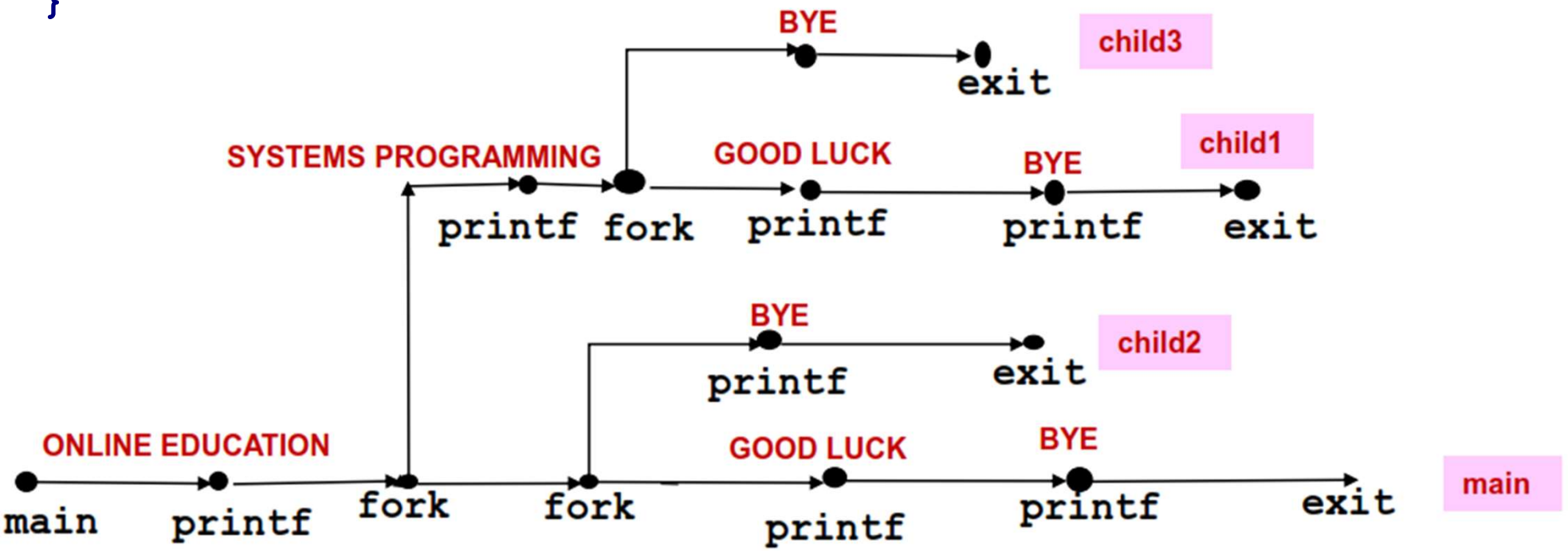
```
int main(){
    printf("L0\n");
    if (fork() != 0)
    {
        printf("L1\n");
        if (fork() != 0)
        {
            printf("L2\n");
        }
    }
    printf("Bye\n");
}
```



HOMEWORK: Nested forks in parent

Draw the corresponding **Process Graph** for the program clearly showing the parent and child processes execution from creation to termination. Provide all the outputs of the parent and child processes

```
int main(){
    printf("HI");
    if (fork() == 0 )
        printf("SYSTEMS PROGRAMMING");
            if (fork() != 0)
                printf("GOOD LUCK");
    printf("BYE \n");
}
```





REAPING CHILD PROCESSES

Reaping terminated children — why, how, and what can go wrong **Zombies & waitpid()**

Zombie processes

When a child terminates, the kernel doesn't delete it immediately. It keeps the exit status and other bookkeeping around until the **parent** collects ("reaps") it. Until then, the dead process is a **zombie** — half alive, half dead — still consuming a PID and a slot in the process table.



What if the parent never reaps? If the parent dies first, the orphaned child is adopted by `init` (pid 1), which reaps it automatically. Explicit reaping only matters for **long-running processes** like shells and servers.

waitpid()

```
#include <sys/types.h>
#include <sys/wait.h>
```

The parent reaps a child by calling `waitpid()`. It suspends the caller until a child in the *wait set* terminates (or has already terminated).

```
pid_t waitpid(pid_t pid, int *statusp, int options);
```

`> 0`	PID of the reaped child
`0`	No child terminated yet
`-1`	Error (check `errno`)

Which children?

The wait set (1st argument)

The `pid` argument controls which children you're waiting for:

PID VALUE	WAIT SET
`> 0`	Just the single child whose PID equals `pid`
`-1`	Any child (equivalent to `wait()`)

These constants are defined in <sys/wait.h> (WNOHANG = 1, WUNTRACED = 2).

The options (3rd argument)

`0 (default)`

Block until a child in the wait set terminates.

`WNOHANG`

Return immediately with 0 if no child has terminated yet. Useful for doing work while waiting.

`WUNTRACED`

Also return for children that have been **stopped** (not just terminated).

```
pid_t ret =
waitpid(pid, &st, WUNTRACED);
```

`WNOHANG | WUNTRACED`

Non-blocking + report stopped children. Returns 0 if nothing happened yet.

Status macros (2nd argument)

If `statusp` is non-NULL, `waitpid` writes encoded status info into `*statusp`. Decode it with these macros from `<sys/wait.h>`:

MACRO	WHAT IT TELLS YOU
`WIFEXITED(s)`	True if child exited normally (via `exit()` or `return`)
`WEXITSTATUS(s)`	The exit code (0–255). Only valid if `WIFEXITED` is true
`WIFSIGNALED(s)`	True if child was killed by an uncaught signal

```c
int status;
pid_t child = waitpid(pid, &status, 0);

if (WIFEXITED(status)){
    printf("child %d exited with code %d\n", child, WEXITSTATUS(status));
}
else if (WIFSIGNALED(status)){
    printf("child %d killed by signal\n", child);
}
```

ECHILD: The calling process has no children to wait for.

EINTR: The call was interrupted by a signal before a child terminated.

When waitpid() fails

To check these, include <errno.h> and inspect errno after waitpid returns -1.

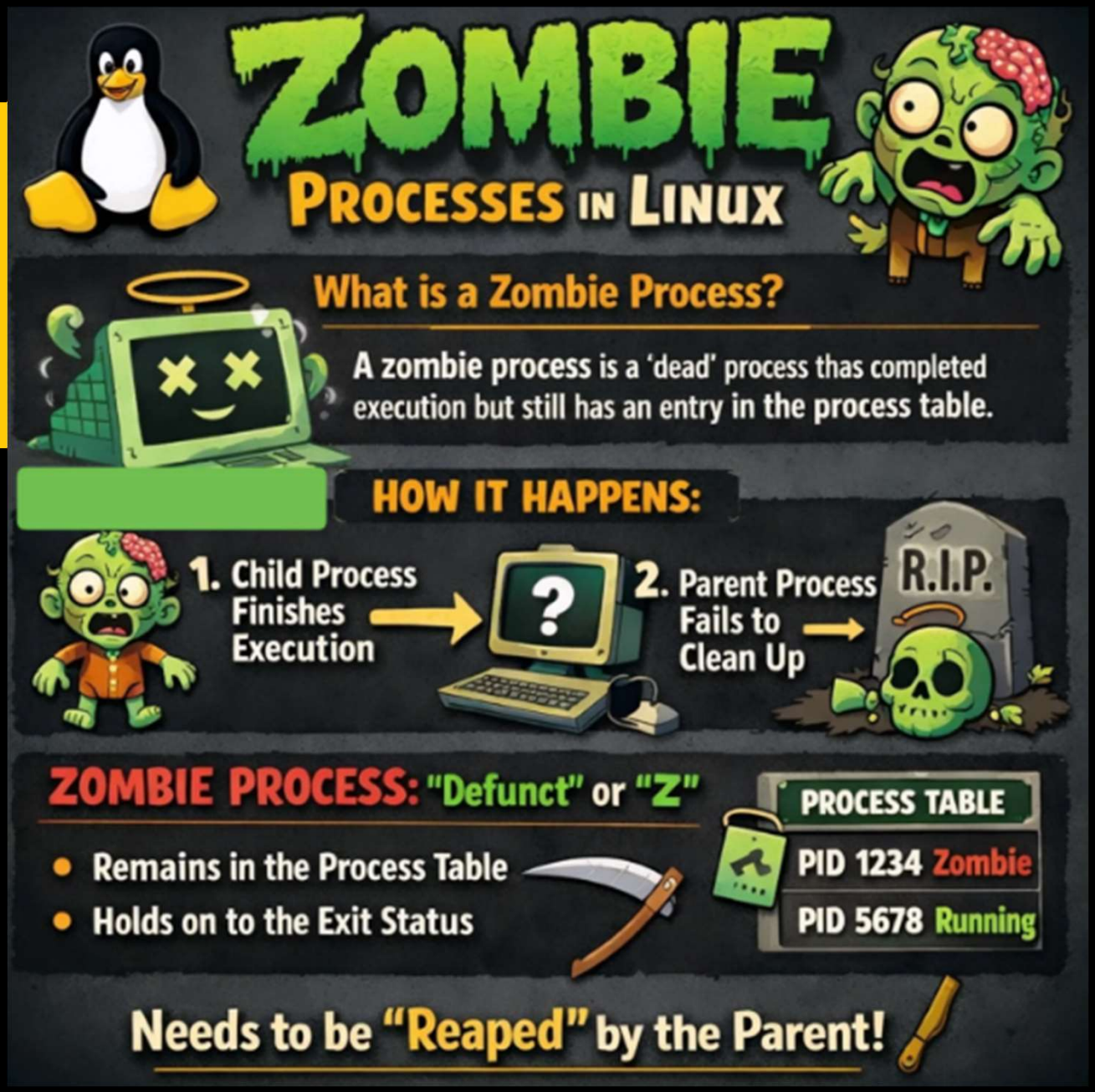
See the zombie lifecycle

In `ps` output you'll see:

- `Z` in the `STAT` column — the process state code for zombie
- `[defunct]` next to the command name

Watch a child become a zombie, then get reaped:

```
$ ./zombie_demo
[parent 8875] forked child 8876
[child 8876] running... doing work
[child 8876] calling exit(0)
[kernel] child 8876 is now a ZOMBIE
[parent 8875] calling waitpid(8876, &status, 0)
[parent 8875] ...blocked, waiting...
[parent 8875] waitpid returned 8876
[parent 8875] WIFEXITED → true, WEXITSTATUS → 0
[kernel] zombie 8876 fully removed ✓
$
```


ZOMBIE
PROCESSES IN LINUX
What is a Zombie Process?
A zombie process is a 'dead' process thas completed execution but still has an entry in the process table.
HOW IT HAPPENS:
1. Child Process Finishes Execution
2. Parent Process Fails to Clean Up
R.I.P.
ZOMBIE PROCESS: "Defunct" or "Z"
Remains in the Process Table
Holds on to the Exit Status
PROCESS TABLE
PID 1234 Zombie
PID 5678 Running
Needs to be "Reaped" by the Parent!



Questions?